

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

УТВЕРЖДАЮ

Зав.кафедрой,

доцент, к. ф.-м. н.

_____ М. В. Огнева

ОТЧЕТ О ПРАКТИКЕ

студентки 3 курса 341 группы

факультета КНиИТ

Хрустицкой Софьи Кирилловны

вид практики: проектная практика (учебная, рассредоточенная)

кафедра: информатики и программирования

курс: 3

семестр: 6

продолжительность: 16 нед., с 05.02.2026 г. по 31.05.2026 г.

Руководитель практики от университета,

доцент, к. п. н.

А. П. Грецова

Саратов 2026

СОДЕРЖАНИЕ

1	Минимальные требования для класса Граф	3
2	Список смежности Ia	14
3	Список смежности Ia	17
4	Список смежности Ib: несколько графов	19
5	Обходы графа II	22
6	Обходы графа II	25
7	Каркас III	28
8	Весы IV а	31
9	Весы IV б	34
10	Весы IV с	37
11	Максимальный поток V	40
12	Задание на визуализацию	44

1 Минимальные требования для класса Граф

Условие. Для решения всех задач курса необходимо создать класс (или иерархию классов — на усмотрение разработчика), содержащий:

1. Структуру для хранения списка смежности графа (не работать с графом через матрицы смежности, если в некоторых алгоритмах удобнее использовать список ребер — реализовать метод, создающий список рёбер на основе списка смежности).
2. Конструкторы (не менее 3-х):
 - конструктор по умолчанию, создающий пустой граф;
 - конструктор, заполняющий данные графа из файла;
 - конструктор-копию (аккуратно, не все сразу делают именно копию);
 - специфические конструкторы для удобства тестирования.
3. Методы:
 - добавляющие вершину;
 - добавляющие ребро (дугу);
 - удаляющие вершину;
 - удаляющие ребро (дугу);
 - выводящие список смежности в файл (в том числе в пригодном для чтения конструктором формате).

Не выполняйте некорректные операции, сообщайте об ошибках.

4. Должны поддерживаться как ориентированные, так и неориентированные графы. Заранее предусмотрите возможность добавления меток и/или весов для дуг. Поддержка мультиграфа не требуется.
5. Добавьте минималистичный консольный интерфейс пользователя (не смешивая его с реализацией!), позволяющий добавлять и удалять вершины и рёбра (дуги) и просматривать текущий список смежности графа.
6. Сгенерируйте не менее 4 входных файлов с разными типами графов (балансируйте на комбинации ориентированность-взвешенность) для тести-

рования класса в этом и последующих заданиях. Графы должны содержать не менее 7-10 вершин, в том числе петли и изолированные вершины.

Замечание:

В зависимости от выбранного способа хранения графа могут появиться дополнительные трудности при удалении-добавлении, например, необходимость переименования вершин, если граф хранится списком (например, `vector` C++, `List` C#). Этого можно избежать, если хранить в списке пару (имя вершины, список смежных вершин), или хранить в другой структуре (например, `Dictionary` C#, `map` в C++, при этом список смежности вершины может также храниться в виде словаря с ключами — смежными вершинами и значениями — весами соответствующих ребер). Идеально, если в качестве вершины реализуется обобщенный тип (`generic`), но достаточно использовать строковый тип или свой класс.

Решение. В проекте реализован обобщённый класс `Graph<T>`. Вершины имеют тип `T`, а список смежности хранится в словаре: ключ верхнего уровня — вершина, значение — словарь смежных вершин и весов рёбер. Такой формат позволяет не переименовывать вершины при удалении и сразу хранить веса.

```
1 public struct Edge<T>
2 {
3     public T From { get; }
4     public T To { get; }
5     public double Weight { get; }
6
7     public Edge(T from, T to, double weight = 1.0)
8     {
9         From = from;
10        To = to;
11        Weight = weight;
12    }
13
14    public override string ToString()
15    {
16        return $"{From} -> {To} : {Weight}";
17    }
18 }
```

```

19
20 public class Graph<T> where T : notnull, IComparable<T>
21 {
22     private Dictionary<T, Dictionary<T, double>> _adjacencyList;
23
24     public bool IsDirected { get; }
25     public bool IsWeighted { get; }
26 }

```

В классе есть конструктор по умолчанию, конструктор-копия, конструктор из файла и дополнительный конструктор по набору вершин.

```

1 public Graph(bool isDirected = false, bool isWeighted = false)
2 {
3     IsDirected = isDirected;
4     IsWeighted = isWeighted;
5     _adjacencyList = new Dictionary<T, Dictionary<T, double>>();
6 }
7
8 public Graph(Graph<T> other)
9 {
10     if (other == null) throw new ArgumentNullException(nameof(other));
11
12     IsDirected = other.IsDirected;
13     IsWeighted = other.IsWeighted;
14     _adjacencyList = new Dictionary<T, Dictionary<T, double>>();
15
16     foreach (var vertex in other._adjacencyList)
17     {
18         var neighbors = new Dictionary<T, double>(vertex.Value);
19         _adjacencyList.Add(vertex.Key, neighbors);
20     }
21 }
22
23 public Graph(string filePath, Func<string, T> parser, Action<string>? log =
    ↪ null)
24 {
25     if (!File.Exists(filePath))
26         throw new FileNotFoundException("Файл не найден", filePath);
27
28     var lines = File.ReadAllLines(filePath)

```

```

29         .Where(l => !string.IsNullOrEmpty(l))
30         .ToArray();
31
32     var headerParts = lines[0].Split(' ',
    ↪ StringSplitOptions.RemoveEmptyEntries);
33     IsDirected = headerParts.Contains("DIRECTED",
    ↪ StringComparer.OrdinalIgnoreCase);
34     IsWeighted = headerParts.Contains("WEIGHTED",
    ↪ StringComparer.OrdinalIgnoreCase);
35
36     _adjacencyList = new Dictionary<T, Dictionary<T, double>>();
37
38     foreach (var vStr in lines[1].Split(' ',
    ↪ StringSplitOptions.RemoveEmptyEntries))
39         AddVertex(parser(vStr));
40 }
41
42 public Graph(IEnumerable<T> vertices, bool isDirected, bool isWeighted)
43     : this(isDirected, isWeighted)
44 {
45     foreach (var v in vertices)
46         AddVertex(v);
47 }

```

Основные операции проверяют корректность действия. Если вершина отсутствует или ребро уже существует, операция не выполняется и возвращается ошибка. Для неориентированного графа второе направление ребра добавляется и удаляется автоматически.

```

1 public bool AddVertex(T vertex)
2 {
3     if (_adjacencyList.ContainsKey(vertex))
4         return false;
5
6     _adjacencyList[vertex] = new Dictionary<T, double>();
7     return true;
8 }
9
10 public bool RemoveVertex(T vertex)
11 {

```

```

12     if (!_adjacencyList.ContainsKey(vertex))
13         return false;
14
15     _adjacencyList.Remove(vertex);
16
17     foreach (var v in _adjacencyList)
18         v.Value.Remove(vertex);
19
20     return true;
21 }
22
23 public void AddEdge(T from, T to, double weight = 1.0)
24 {
25     if (!_adjacencyList.ContainsKey(from))
26         throw new ArgumentException($"Вершина {from} не существует.");
27     if (!_adjacencyList.ContainsKey(to))
28         throw new ArgumentException($"Вершина {to} не существует.");
29
30     if (!IsWeighted) weight = 1.0;
31
32     if (_adjacencyList[from].ContainsKey(to))
33         throw new
34 ↪ InvalidOperationException($"Ребро из {from} в {to} уже существует.");
35
36     _adjacencyList[from][to] = weight;
37
38     if (!IsDirected && !from.Equals(to))
39         _adjacencyList[to][from] = weight;
40 }
41
42 public bool RemoveEdge(T from, T to)
43 {
44     if (!_adjacencyList.ContainsKey(from)) return false;
45
46     bool removed = _adjacencyList[from].Remove(to);
47
48     if (!IsDirected && removed && !from.Equals(to) &&
49 ↪ _adjacencyList.ContainsKey(to))
50         _adjacencyList[to].Remove(from);
51
52     return removed;
53 }

```

51 }

Для алгоритмов, которым удобнее работать со списком рёбер, реализован метод `GetEdgeList`. В неориентированном графе каждое ребро попадает в список один раз.

```
1 public List<Edge<T>> GetEdgeList()
2 {
3     var edges = new List<Edge<T>>();
4
5     foreach (var kvp in _adjacencyList)
6     {
7         T from = kvp.Key;
8         foreach (var innerKvp in kvp.Value)
9         {
10             T to = innerKvp.Key;
11             double w = innerKvp.Value;
12
13             if (!IsDirected)
14             {
15                 if (from.CompareTo(to) <= 0)
16                     edges.Add(new Edge<T>(from, to, w));
17             }
18             else
19             {
20                 edges.Add(new Edge<T>(from, to, w));
21             }
22         }
23     }
24
25     return edges;
26 }
```

Сохранение поддерживает два формата: список рёбер, пригодный для чтения конструктором, и список смежности для просмотра.

```
1 public void SaveToFile(string filePath, GraphSaveFormat format =
    ↳ GraphSaveFormat.EdgeList)
2 {
3     using (var writer = new StreamWriter(filePath))
```



```

4      {
5          string type = IsDirected ? "DIRECTED" : "UNDIRECTED";
6          string weighted = IsWeighted ? "WEIGHTED" : "UNWEIGHTED";
7          writer.WriteLine($"{type} {weighted}");
8
9          writer.WriteLine(string.Join(" ", _adjacencyList.Keys));
10
11         if (format == GraphSaveFormat.EdgeList)
12         {
13             foreach (var edge in GetEdgeList())
14             {
15                 string line = $"{edge.From} {edge.To}";
16                 if (IsWeighted)
17                     line += $" {edge.Weight}";
18                 writer.WriteLine(line);
19             }
20         }
21     }
22 }

```

Пользовательский интерфейс вынесен в отдельный класс ConsoleInterface, поэтому логика хранения графа и работа с консолью не смешиваются.

```

1 public class ConsoleInterface
2 {
3     private Graph<string> _graph;
4
5     public ConsoleInterface()
6     {
7         _graph = new Graph<string>(isDirected: false, isWeighted: false);
8     }
9
10    public void Run()
11    {
12        while (true)
13        {
14            Console.WriteLine("1. Создать новый граф (пустой)");
15            Console.WriteLine("2. Загрузить граф из файла");
16            Console.WriteLine("3. Добавить вершину");
17            Console.WriteLine("4. Удалить вершину");
18            Console.WriteLine("5. Добавить ребро (дугу)");

```

```

19         Console.WriteLine("6. Удалить ребро (дугу)");
20         Console.WriteLine("7. Показать список смежности");
21         Console.WriteLine("20. Выход");
22         Console.Write("Выберите опцию: ");
23
24         string choice = Console.ReadLine();
25         // далее выполняется выбранная команда
26     }
27 }
28 }

```

Входные данные.

Для тестирования используются файлы в формате: первая строка задаёт тип графа, вторая — вершины, следующие строки — рёбра. Пример входного файла для ориентированного взвешенного графа:

```

DIRECTED WEIGHTED
1 2 3 4 5 6 7
1 2 1
1 3 1
2 4 1
3 4 1
2 5 2
3 6 2
4 7 2
5 7 1
6 7 1

```

Пример интерфейса в консоли.

После запуска программы открывается консольное меню. Пользователь выбирает действие по номеру: можно создать новый граф, загрузить его из файла, добавить или удалить вершину, добавить или удалить ребро, посмотреть текущий список смежности и сохранить граф. Ниже приведён пример работы с интерфейсом: создаётся ориентированный взвешенный граф, добавляются вершины и дуги, затем выводится список смежности.

--- Консольный Интерфейс Графа ---

1. Создать новый граф (пустой)
2. Загрузить граф из файла
3. Добавить вершину
4. Удалить вершину
5. Добавить ребро (дугу)
6. Удалить ребро (дугу)
7. Показать список смежности
8. Вывести не смежные вершины
9. Вывести изолированные вершины
10. Удалить ребра висячие вершины
11. Найти фундаментальные циклы (DFS)
12. Найти фундаментальные циклы (BFS)
13. Найти вершину с равными путями (BFS)
14. Найти минимальное остовное дерево (Boruvka)
15. Найти кратчайший путь (Dijkstra)
16. Найти K кратчайших путей (Bellman-Ford)
17. Найти циклы отрицательного веса (Floyd-Warshall)
18. Найти максимальный поток (Edmonds-Karp)
19. Сохранить в файл
20. Выход

Выберите опцию: 1

Ориентированный? (y/n): y

Взвешенный? (y/n): y

Новый пустой граф создан.

При добавлении вершины программа запрашивает её имя. Если вершины ещё нет в графе, она добавляется; если такая вершина уже существует, выводится сообщение об ошибке.

Выберите опцию: 3

Введите имя вершины: A

Вершина 'A' добавлена.

Выберите опцию: 3

Введите имя вершины: B

Вершина 'B' добавлена.

Выберите опцию: 3

Введите имя вершины: C

Вершина 'C' добавлена.

Выберите опцию: 3

Введите имя вершины: A

Вершина 'A' уже существует.

При добавлении дуги интерфейс запрашивает начальную вершину, конечную вершину и вес, так как граф был создан взвешенным. Если указать несуществующую вершину или попытаться добавить уже существующую дугу, операция не выполняется.

Выберите опцию: 5

Введите исходную вершину: A

Введите конечную вершину: B

Введите вес: 4

Ребро добавлено.

Выберите опцию: 5

Введите исходную вершину: A

Введите конечную вершину: C

Введите вес: 2

Ребро добавлено.

Выберите опцию: 5

Введите исходную вершину: A

Введите конечную вершину: B

Введите вес: 5

Ошибка: Ребро из A в B уже существует.

Команда просмотра списка смежности выводит тип графа, количество вершин и список соседей каждой вершины. Для взвешенного графа рядом с каждой смежной вершиной указывается вес дуги.

Выберите опцию: 7

Граф (Ориентированный: Да, Взвешенный: Да)

Количество вершин: 3

A: B(4), C(2)

B: (изолированная)

C: (изолированная)

Удаление ребра и вершины также сопровождается сообщением о результате операции. После удаления вершины все дуги, ведущие в неё, удаляются из списков смежности остальных вершин.

Выберите опцию: 6

Введите исходную вершину: A

Введите конечную вершину: C

Ребро удалено.

Выберите опцию: 4

Введите имя вершины: B

Вершина 'B' удалена.

Выберите опцию: 7

Граф (Ориентированный: Да, Взвешенный: Да)

Количество вершин: 2

A: (изолированная)

C: (изолированная)

2 Список смежности Ia

Условие. Вывести все вершины графа, не смежные с данной.

Решение. Метод получает вершину, проверяет её наличие и сравнивает множество всех вершин со словарём соседей выбранной вершины. Сама вершина в результат не включается.

```
1 public List<T> GetNonAdjacentVertices(T vertex)
2 {
3     if (!_adjacencyList.ContainsKey(vertex))
4         throw new ArgumentException($"Вершина {vertex} не найдена.");
5
6     var nonAdjacent = new List<T>();
7     var neighbors = _adjacencyList[vertex];
8
9     foreach (var v in _adjacencyList.Keys)
10    {
11        if (!v.Equals(vertex) && !neighbors.ContainsKey(v))
12            nonAdjacent.Add(v);
13    }
14
15    return nonAdjacent;
16 }
```

В консольном интерфейсе пользователь вводит имя вершины, после чего результат выводится строкой.

```
1 private void ShowNonAdjacent()
2 {
3     Console.Write("Введите имя вершины: ");
4     string v = Console.ReadLine();
5
6     if (!_graph.ContainsVertex(v))
7     {
8         Console.WriteLine($"Вершина '{v}' не найдена.");
9         return;
10    }
11
12    var nonAdjacent = _graph.GetNonAdjacentVertices(v);
```

```

13     Console.WriteLine($"Вершины, не смежные с '{v}': {string.Join(",
↪     ", nonAdjacent)}}");
14 }

```

Пример входных данных. Для проверки использовался файл /Users /lunis/Documents/graph1.txt.

```

UNDIRECTED UNWEIGHTED
A B C D E F G H
A B
B C
C A
D D
F G
G H
C F
F H
A G

```

Пример работы в консоли. В консольном интерфейсе граф загружается из файла, затем выбирается пункт 8 и вводится вершина A.

```

--- Консольный Интерфейс Графа ---
1. Создать новый граф (пустой)
2. Загрузить граф из файла
3. Добавить вершину
4. Удалить вершину
5. Добавить ребро (дугу)
6. Удалить ребро (дугу)
7. Показать список смежности
8. Вывести не смежные вершины
9. Вывести изолированные вершины
10. Удалить ребра висячие вершины
11. Найти фундаментальные циклы (DFS)
12. Найти фундаментальные циклы (BFS)
13. Найти вершину с равными путями (BFS)
14. Найти минимальное остовное дерево (Boruvka)
15. Найти кратчайший путь (Dijkstra)
16. Найти K кратчайших путей (Bellman-Ford)
17. Найти циклы отрицательного веса (Floyd-Warshall)

```

18. Найти максимальный поток (Edmonds-Karp)

19. Сохранить в файл

20. Выход

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph1.txt

Граф успешно загружен.

--- Консольный Интерфейс Графа ---

1. Создать новый граф (пустой)

2. Загрузить граф из файла

3. Добавить вершину

4. Удалить вершину

5. Добавить ребро (дугу)

6. Удалить ребро (дугу)

7. Показать список смежности

8. Вывести не смежные вершины

9. Вывести изолированные вершины

10. Удалить ребра висячие вершины

11. Найти фундаментальные циклы (DFS)

12. Найти фундаментальные циклы (BFS)

13. Найти вершину с равными путями (BFS)

14. Найти минимальное остовное дерево (Boruvka)

15. Найти кратчайший путь (Dijkstra)

16. Найти K кратчайших путей (Bellman-Ford)

17. Найти циклы отрицательного веса (Floyd-Warshall)

18. Найти максимальный поток (Edmonds-Karp)

19. Сохранить в файл

20. Выход

Выберите опцию: 8

Введите имя вершины: A

Вершины, не смежные с 'A': D, E, F, H

3 Список смежности Ia

Условие. Вывести все изолированные вершины орграфа, то есть вершины степени 0.

Решение. Для ориентированного графа учитываются и исходящие, и входящие дуги. Сначала собирается множество вершин, в которые входят дуги, затем выбираются вершины без исходящих и входящих дуг.

```
1 public List<T> GetIsolatedVertices()
2 {
3     var isolated = new List<T>();
4     var allVertices = _adjacencyList.Keys.ToList();
5
6     var incomingEdges = new HashSet<T>();
7     foreach (var kvp in _adjacencyList)
8     {
9         foreach (var neighbor in kvp.Value.Keys)
10             incomingEdges.Add(neighbor);
11     }
12
13     foreach (var v in allVertices)
14     {
15         bool hasOutgoing = _adjacencyList[v].Count > 0;
16
17         if (IsDirected)
18         {
19             bool hasIncoming = incomingEdges.Contains(v);
20             if (!hasOutgoing && !hasIncoming)
21                 isolated.Add(v);
22         }
23         else
24         {
25             if (!hasOutgoing)
26                 isolated.Add(v);
27         }
28     }
29
30     return isolated;
31 }
```

Вывод в интерфейсе:

```
1 private void ShowIsolated()
2 {
3     var isolated = _graph.GetIsolatedVertices();
4     if (isolated.Count == 0)
5         Console.WriteLine("В графе нет изолированных вершин.");
6     else
7         Console.WriteLine($"Изолированные вершины: {string.Join(",
↪ ", isolated)}}");
8 }
```

Пример входных данных. Для проверки использовался файл /Users /lunis/Documents/graph2.txt.

```
DIRECTED WEIGHTED
1 2 3 4 5 6 7
1 2 1.5
2 3 2.0
3 1 0.5
4 5 10
5 5 1
7 1 3
```

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph2.txt

Граф успешно загружен.

Выберите опцию: 9

Изолированные вершины: 6

4 Список смежности Iб: несколько графов

Условие. Построить граф, полученный удалением рёбер, ведущих в висячие вершины.

Решение. Метод сначала создаёт копию исходного графа, чтобы не изменять его во время вычисления степеней. Затем вычисляются степени вершин, находятся висячие вершины степени 1 и удаляются все рёбра, которые в них входят.

```
1 public Graph<T> RemoveEdgesToPendantVertices()
2 {
3     var newGraph = new Graph<T>(this);
4
5     var degrees = new Dictionary<T, int>();
6     foreach (var v in _adjacencyList.Keys)
7         degrees[v] = 0;
8
9     foreach (var kvp in _adjacencyList)
10    {
11        T u = kvp.Key;
12        degrees[u] += kvp.Value.Count;
13
14        foreach (var v in kvp.Value.Keys)
15        {
16            if (IsDirected)
17            {
18                if (!degrees.ContainsKey(v)) degrees[v] = 0;
19                degrees[v]++;
20            }
21        }
22    }
23
24    if (!IsDirected)
25    {
26        foreach (var v in _adjacencyList.Keys)
27            degrees[v] = _adjacencyList[v].ContainsKey(v)
28                ? _adjacencyList[v].Count + 1
29                : _adjacencyList[v].Count;
30    }
```

```

31
32     var pendantVertices = degrees
33         .Where(kvp => kvp.Value == 1)
34         .Select(kvp => kvp.Key)
35         .ToHashSet();
36
37     var edgesToRemove = new List<(T From, T To)>();
38
39     foreach (var kvp in _adjacencyList)
40         foreach (var v in kvp.Value.Keys)
41             if (pendantVertices.Contains(v))
42                 edgesToRemove.Add((kvp.Key, v));
43
44     foreach (var edge in edgesToRemove)
45         newGraph.RemoveEdge(edge.From, edge.To);
46
47     return newGraph;
48 }

```

В интерфейсе результат становится новым текущим графом.

```

1 private void RemoveEdgesToPendant()
2 {
3     var newGraph = _graph.RemoveEdgesToPendantVertices();
4     _graph = newGraph;
5     Console.WriteLine(
6         "Ребра, ведущие в висячие вершины, удалены. " +
7         "Граф обновлен.");
8 }

```

Пример входных данных. Для проверки использовался файл /Users /lunis/Documents/graph4.txt.

```

DIRECTED UNWEIGHTED
v1 v2 v3 v4 v5 v6 v7
v1 v2
v2 v3
v3 v4
v4 v1
v5 v6
v6 v7
v1 v1

```

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph4.txt

Граф успешно загружен.

Выберите опцию: 10

Ребра, ведущие в висячие вершины, удалены. Граф обновлен.

Выберите опцию: 7

Граф (Ориентированный: Да, Взвешенный: Нет)

Количество вершин: 7

v1: v2, v1

v2: v3

v3: v4

v4: v1

v5: v6

v6: (изолированная)

v7: (изолированная)

5 Обходы графа II

Условие. Найти фундаментальное множество циклов орграфа.

Решение. В программе реализован поиск циклов при обходе в глубину. Используются множества посещённых вершин и текущего рекурсивного стека. Если во время обхода находится дуга в вершину из стека, восстанавливается цикл по словарю родителей.

```
1 public List<List<T>> GetFundamentalCyclesDFS(Action<string>? log = null)
2 {
3     var cycles = new List<List<T>>();
4     var visited = new HashSet<T>();
5     var recursionStack = new HashSet<T>();
6     var parent = new Dictionary<T, T>();
7
8     foreach (var vertex in _adjacencyList.Keys)
9     {
10         if (!visited.Contains(vertex))
11             DFSFindCycles(vertex, visited, recursionStack, parent, cycles, log);
12     }
13
14     return cycles;
15 }
```

Основная часть восстановления цикла:

```
1 private void DFSFindCycles(
2     T current,
3     HashSet<T> visited,
4     HashSet<T> recursionStack,
5     Dictionary<T, T> parent,
6     List<List<T>> cycles,
7     Action<string>? log)
8 {
9     visited.Add(current);
10    recursionStack.Add(current);
11    log?.Invoke($"DFS: посещаем {current}");
12
13    foreach (var neighbor in _adjacencyList[current].Keys)
14    {
```

```

15         log?.Invoke($"DFS: ребро {current} -> {neighbor}");
16
17         if (recursionStack.Contains(neighbor))
18         {
19             var cycle = new List<T>();
20             var p = current;
21
22             while (!p.Equals(neighbor))
23             {
24                 cycle.Add(p);
25                 p = parent[p];
26             }
27
28             cycle.Add(neighbor);
29             cycle.Reverse();
30             cycle.Add(neighbor);
31             cycles.Add(cycle);
32
33             log?.Invoke($"DFS: найден цикл {string.Join(" -> ", cycle)}");
34         }
35         else if (!visited.Contains(neighbor))
36         {
37             parent[neighbor] = current;
38             DFSFindCycles(neighbor, visited, recursionStack, parent, cycles,
↪ log);
39         }
40     }
41
42     recursionStack.Remove(current);
43 }

```

Пример входных данных. Для проверки использовался файл /Users /lunis/Documents/graph4.txt.

```

DIRECTED UNWEIGHTED
v1 v2 v3 v4 v5 v6 v7
v1 v2
v2 v3
v3 v4
v4 v1
v5 v6

```

v6 v7

v1 v1

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph4.txt

Граф успешно загружен.

Выберите опцию: 11

Найдено циклов (DFS): 2

v1 -> v2 -> v3 -> v4 -> v1

v1 -> v1

6 Обходы графа II

Условие. В данном графе найти такую вершину, в которую есть пути как из вершины u , так и из вершины v одинаковой длины по числу рёбер.

Решение. Для решения реализованы два варианта. Вариант BFS вычисляет расстояния от обеих стартовых вершин и ищет первую вершину, для которой расстояния совпали.

```
1 public T FindEquidistantVertexBFS(T u, T v)
2 {
3     if (!_adjacencyList.ContainsKey(u))
4         throw new ArgumentException($"Вершина {u} не найдена.");
5     if (!_adjacencyList.ContainsKey(v))
6         throw new ArgumentException($"Вершина {v} не найдена.");
7
8     var distU = GetDistancesBFS(u);
9     var distV = GetDistancesBFS(v);
10
11     foreach (var node in _adjacencyList.Keys)
12         if (distU.ContainsKey(node) && distV.ContainsKey(node) && distU[node] ==
13             ↪ distV[node])
14             return node;
15     return default;
16 }
17
18 private Dictionary<T, int> GetDistancesBFS(T start)
19 {
20     var dist = new Dictionary<T, int>();
21     var queue = new Queue<T>();
22
23     dist[start] = 0;
24     queue.Enqueue(start);
25
26     while (queue.Count > 0)
27     {
28         var curr = queue.Dequeue();
29         foreach (var neighbor in _adjacencyList[curr].Keys)
30         {
31             if (!dist.ContainsKey(neighbor))
```

```

32         {
33             dist[neighbor] = dist[curr] + 1;
34             queue.Enqueue(neighbor);
35         }
36     }
37 }
38
39 return dist;
40 }

```

Вариант DFS собирает глубины достижимых вершин из u , а затем ищет совпадение при обходе из v .

```

1 public T FindEquidistantVertexDFS(T u, T v)
2 {
3     var depthU = new Dictionary<T, int>();
4     DFSCollectDepths(u, 0, new HashSet<T>(), depthU);
5
6     return DFSCheckDepths(v, 0, new HashSet<T>(), depthU);
7 }
8
9 private void DFSCollectDepths(T current, int depth, HashSet<T> visited,
    ↪ Dictionary<T, int> depths)
10 {
11     visited.Add(current);
12     if (!depths.ContainsKey(current))
13         depths[current] = depth;
14
15     foreach (var neighbor in _adjacencyList[current].Keys)
16         if (!visited.Contains(neighbor))
17             DFSCollectDepths(neighbor, depth + 1, visited, depths);
18 }

```

Пример входных данных. Для проверки использовался файл `/Users/lunis/Documents/graph6.txt`.

```

DIRECTED WEIGHTED
1 2 3 4 5 6 7
1 2 1
1 3 1

```

2 4 1
3 4 1
2 5 2
3 6 2
4 7 2
5 7 1
6 7 1

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph6.txt

Граф успешно загружен.

Выберите опцию: 13

Введите вершину U: 2

Введите вершину V: 3

[BFS] Найдена вершина: 4

[DFS] Найдена вершина: 4

7 Каркас III

Условие. Дан взвешенный неориентированный граф из n вершин и m рёбер. Требуется найти в нём каркас минимального веса алгоритмом Борувки.

Решение. Метод проверяет, что граф неориентированный, создаёт пустой граф для результата и использует структуру непересекающихся множеств через словари `parent` и `rank`.

```
1 public Graph<T> GetMinimumSpanningTreeBoruvka(Action<string>? log = null)
2 {
3     if (IsDirected)
4         throw new InvalidOperationException(
5             "Алгоритм Борувка работает только для неориентированных графов.");
6
7     var mst = new Graph<T>(false, IsWeighted);
8
9     foreach (var v in _adjacencyList.Keys)
10         mst.AddVertex(v);
11
12     var parent = new Dictionary<T, T>();
13     var rank = new Dictionary<T, int>();
14
15     foreach (var v in _adjacencyList.Keys)
16     {
17         parent[v] = v;
18         rank[v] = 0;
19     }
20 }
```

На каждом шаге для каждой компоненты выбирается самое дешёвое исходящее ребро. Если оно соединяет разные компоненты, ребро добавляется в минимальный каркас.

```
1 while (numTrees > 1)
2 {
3     var cheapest = new Dictionary<T, (T u, T v, double w)>();
4     bool edgeAdded = false;
5
6     foreach (var u in _adjacencyList.Keys)
```

```

7      {
8          foreach (var kvp in _adjacencyList[u])
9          {
10             T v = kvp.Key;
11             double w = kvp.Value;
12
13             if (!IsDirected && u.CompareTo(v) > 0)
14                 continue;
15
16             T setU = Find(u);
17             T setV = Find(v);
18
19             if (!setU.Equals(setV))
20             {
21                 if (!cheapest.ContainsKey(setU) || w < cheapest[setU].w)
22                     cheapest[setU] = (u, v, w);
23
24                 if (!cheapest.ContainsKey(setV) || w < cheapest[setV].w)
25                     cheapest[setV] = (u, v, w);
26             }
27         }
28     }
29
30     foreach (var edge in cheapest.Values.Distinct().ToList())
31     {
32         T setU = Find(edge.u);
33         T setV = Find(edge.v);
34
35         if (!setU.Equals(setV))
36         {
37             mst.AddEdge(edge.u, edge.v, edge.w);
38             Union(setU, setV);
39             numTrees--;
40             edgeAdded = true;
41         }
42     }
43
44     if (!edgeAdded)
45         break;
46 }

```

Пример входных данных. Для проверки использовался файл /Users/lunis/Documents/graph3.txt.

```
UNDIRECTED WEIGHTED
X Y Z W P Q R S
X Y 5
Y Z 3
X Z 10
W W 2
P Q 1
Q R 1
R P 1
```

Пример работы в консоли.

Выберите опцию: 2
Введите путь к файлу: /Users/lunis/Documents/graph3.txt
Граф успешно загружен.

Выберите опцию: 14
Нет рёбер для объединения. Граф несвязный.

=== Готовое минимальное остовное дерево ===

```
X -> Y : 5
Y -> Z : 3
P -> Q : 1
P -> R : 1
```

Минимальное остовное дерево (Алгоритм Борувки):

```
X -> Y : 5
Y -> Z : 3
P -> Q : 1
P -> R : 1
```

Общий вес: 10

8 Веса IV а

Условие. В графе нет рёбер отрицательного веса. Алгоритмом Дейкстры найти длину кратчайшего пути из u в v и вывести все пути такой длины.

Решение. Реализация сначала проверяет отсутствие отрицательных весов. Для каждой вершины хранится расстояние и множество предшественников, поэтому при равных расстояниях сохраняются все варианты кратчайших путей.

```
1 public (double distance, List<List<T>> paths)
2     GetShortestPathsDijkstra(T start, T end, Action<string>? log = null)
3 {
4     foreach (var u in _adjacencyList)
5         foreach (var v in u.Value)
6             if (v.Value < 0)
7                 throw new InvalidOperationException(
8
9     → "Граф содержит ребра с отрицательным весом. Алгоритм Дейкстры неприменим.");
10
11     var distances = new Dictionary<T, double>();
12     var predecessors = new Dictionary<T, HashSet<T>>();
13     var pq = new PriorityQueue<T, double>();
14
15     foreach (var v in _adjacencyList.Keys)
16         distances[v] = double.PositiveInfinity;
17
18     distances[start] = 0;
19     pq.Enqueue(start, 0);
20 }
```

При релаксации, если найдено такое же расстояние, предшественник добавляется в множество. После завершения работы все пути восстанавливаются рекурсивно.

```
1 double newDist = distances[u] + weight;
2
3 if (newDist < distances[v])
4 {
5     distances[v] = newDist;
6     predecessors[v] = new HashSet<T> { u };
```

```

7    pq.Enqueue(v, newDist);
8 }
9 else if (Math.Abs(newDist - distances[v]) < 1e-9)
10 {
11     if (!predecessors.ContainsKey(v))
12         predecessors[v] = new HashSet<T>();
13     predecessors[v].Add(u);
14 }
15
16 var paths = new List<List<T>>>();
17 var pathStack = new List<T>();
18 ReconstructPathsDFS(end, start, predecessors, pathStack, paths);
19
20 return (distances[end], paths);

```

Пример входных данных. Для проверки использовался файл /Users/lunis/Documents/graph6.txt.

DIRECTED WEIGHTED

```

1 2 3 4 5 6 7
1 2 1
1 3 1
2 4 1
3 4 1
2 5 2
3 6 2
4 7 2
5 7 1
6 7 1

```

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph6.txt

Граф успешно загружен.

Выберите опцию: 15

Введите начальную вершину: 1

Введите конечную вершину: 7

Кратчайшее расстояние: 4

Найдено путей: 4

1 -> 2 -> 4 -> 7

1 -> 3 -> 4 -> 7

1 -> 3 -> 6 -> 7

1 -> 2 -> 5 -> 7

9 Веса IV b

Условие. В графе нет циклов отрицательного веса. Алгоритмом Беллмана-Форда найти k кратчайших путей между вершинами u и v .

Решение. Для каждой вершины хранится не одно расстояние, а список до k лучших состояний. Состояние включает стоимость, предшественника, индекс состояния предшественника и набор уже посещённых вершин, чтобы восстанавливались простые пути.

```
1 public List<(double distance, List<T> path)>
2     GetKShortestPathsBellmanFord(T start, T end, int k, Action<string>? log =
    ↪ null)
3 {
4     if (k <= 0)
5         throw new ArgumentException("K должно быть больше 0.");
6
7     var dist = new Dictionary<T, List<(double cost, T pred, int predIdx,
8         HashSet<T> visitedNodes)>>();
9
10    foreach (var v in _adjacencyList.Keys)
11        dist[v] = new List<(double, T, int, HashSet<T>)>();
12
13    var startVisited = new HashSet<T> { start };
14    dist[start].Add((0, default, -1, startVisited));
15 }
```

На каждой итерации просматриваются все рёбра и расширяются уже найденные состояния. Список состояний вершины сортируется по стоимости и обрезается до k .

```
1 for (int i = 0; i < maxIterations; i++)
2 {
3     bool changed = false;
4
5     foreach (var edge in allEdges)
6     {
7         T u = edge.u;
8         T v = edge.v;
```

```

9         double w = edge.w;
10
11         for (int idx = 0; idx < dist[u].Count; idx++)
12         {
13             var currentState = dist[u][idx];
14             if (currentState.visitedNodes.Contains(v))
15                 continue;
16
17             double newCost = currentState.cost + w;
18             var newVisited = new HashSet<T>(currentState.visitedNodes);
19             newVisited.Add(v);
20
21             dist[v].Add((newCost, u, idx, newVisited));
22             dist[v].Sort((a, b) => a.cost.CompareTo(b.cost));
23
24             if (dist[v].Count > k)
25                 dist[v].RemoveRange(k, dist[v].Count - k);
26
27             changed = true;
28         }
29     }
30
31     if (!changed)
32         break;
33 }

```

Пример входных данных. Для проверки использовался файл /Users/lunis/Documents/graph8.txt.

```

DIRECTED WEIGHTED
A B C
A B 2
A C 5
C B -10

```

Пример работы в консоли.

```

Выберите опцию: 2
Введите путь к файлу: /Users/lunis/Documents/graph8.txt
Граф успешно загружен.

```

Выберите опцию: 16

Введите начальную вершину: A

Введите конечную вершину: B

Введите количество путей (K): 2

--- Поиск 2 простых кратчайших путей из A в B (модифицированный Беллман-Форд)

→ ---

Восстановлен путь: A -> C -> B со стоимостью -5

Восстановлен путь: A -> B со стоимостью 2

Найдено 2 кратчайших путей (Bellman-Ford):

#1 (Длина: -5): A -> C -> B

#2 (Длина: 2): A -> B

10 Веса IV с

Условие. В графе могут быть циклы отрицательного веса. Алгоритмом Флойда вывести все отрицательные циклы.

Решение. В методе создаются матрица расстояний и матрица родителей. После выполнения релаксаций Флойда-Уоршелла отрицательный цикл определяется по условию $\text{dist}[i, i] < 0$.

```
1 public List<List<T>> GetNegativeCyclesFloydWarshall(Action<string>? log = null)
2 {
3     var vertices = _adjacencyList.Keys.ToList();
4     int n = vertices.Count;
5     var vertexToIndex = new Dictionary<T, int>();
6
7     for (int i = 0; i < n; i++)
8         vertexToIndex[vertices[i]] = i;
9
10    double[,] dist = new double[n, n];
11    int?[,] parent = new int?[n, n];
12
13    for (int i = 0; i < n; i++)
14    {
15        for (int j = 0; j < n; j++)
16        {
17            dist[i, j] = double.PositiveInfinity;
18            parent[i, j] = null;
19        }
20
21        dist[i, i] = 0;
22    }
23 }
```

Обновление расстояний и поиск циклов:

```
1 for (int k = 0; k < n; k++)
2 {
3     for (int i = 0; i < n; i++)
4     {
5         for (int j = 0; j < n; j++)
6         {
```

```

7         if (!double.IsPositiveInfinity(dist[i, k]) &&
8             !double.IsPositiveInfinity(dist[k, j]) &&
9             dist[i, k] + dist[k, j] < dist[i, j])
10        {
11            dist[i, j] = dist[i, k] + dist[k, j];
12            parent[i, j] = parent[k, j];
13        }
14    }
15 }
16 }
17
18 for (int i = 0; i < n; i++)
19 {
20     if (dist[i, i] < 0)
21     {
22         // по parent восстанавливается отрицательный цикл
23     }
24 }

```

Чтобы один и тот же цикл не выводился несколько раз, он нормализуется и сохраняется в HashSet.

```

1 var minVertex = cycle.Min();
2 int minIdx = cycle.IndexOf(minVertex);
3 var normalizedCycle = new List<T>();
4
5 for (int k = 0; k < cycle.Count - 1; k++)
6     normalizedCycle.Add(cycle[(minIdx + k) % (cycle.Count - 1)]);
7
8 normalizedCycle.Add(normalizedCycle[0]);
9
10 string hash = string.Join(">", normalizedCycle);
11 if (seenCycles.Add(hash))
12     cycles.Add(normalizedCycle);

```

Пример входных данных. Для проверки использовался файл /Users /lunis/Documents/graph7.txt.

```

DIRECTED WEIGHTED
1 2 3 4 5 6 7

```

```
1 2 2
2 3 -5
3 1 1
3 4 2
4 5 2
5 6 -4
6 4 1
6 7 3
1 7 10
```

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph7.txt

Граф успешно загружен.

Выберите опцию: 17

Найдено циклов отрицательного веса: 2

1 -> 2 -> 3 -> 1

4 -> 5 -> 6 -> 4

11 Максимальный поток V

Условие. Решить задачу на нахождение максимального потока любым алгоритмом. Подготовить примеры, демонстрирующие работу алгоритма в разных случаях.

Решение. В проекте реализован алгоритм Эдмондса-Карпа. Вес дуги используется как пропускная способность, поэтому метод применяется к ориентированному взвешенному графу с неотрицательными весами.

```
1 public double GetMaxFlowEdmondsKarp(T source, T sink, Action<string>? log =  
    ↪ null)  
2 {  
3     if (!_adjacencyList.ContainsKey(source))  
4         throw new ArgumentException($"Вершина {source} не найдена.");  
5     if (!_adjacencyList.ContainsKey(sink))  
6         throw new ArgumentException($"Вершина {sink} не найдена.");  
7     if (source.Equals(sink))  
8         return 0;  
9  
10    var residualCapacity = new Dictionary<T, Dictionary<T, double>>();  
11    foreach (var u in _adjacencyList.Keys)  
12        residualCapacity[u] = new Dictionary<T, double>();  
13 }
```

Сначала строится остаточная сеть. Для каждой исходной дуги создаётся обратная дуга с нулевой остаточной пропускной способностью.

```
1 foreach (var u in _adjacencyList)  
2 {  
3     foreach (var v in u.Value)  
4     {  
5         T from = u.Key;  
6         T to = v.Key;  
7         double capacity = v.Value;  
8  
9         if (capacity < 0)  
10            throw new InvalidOperationException(  
11                "Алгоритм Эдмондса-Карпа не поддерживает " +  
12                "отрицательные пропускные способности.");  
13     }
```



```

13
14     if (!residualCapacity[from].ContainsKey(to))
15         residualCapacity[from][to] = 0;
16     residualCapacity[from][to] += capacity;
17
18     if (!residualCapacity.ContainsKey(to))
19         residualCapacity[to] = new Dictionary<T, double>();
20     if (!residualCapacity[to].ContainsKey(from))
21         residualCapacity[to][from] = 0;
22 }
23 }

```

Затем в остаточной сети BFS ищет увеличивающий путь. Найденный поток прибавляется к ответу, а остаточные пропускные способности обновляются.

```

1 double maxFlow = 0;
2
3 while (true)
4 {
5     var parent = new Dictionary<T, T>();
6     var queue = new Queue<T>();
7
8     queue.Enqueue(source);
9     parent[source] = source;
10
11     bool reachedSink = false;
12
13     while (queue.Count > 0 && !reachedSink)
14     {
15         var u = queue.Dequeue();
16
17         foreach (var kvp in residualCapacity[u])
18         {
19             var v = kvp.Key;
20             var capacity = kvp.Value;
21
22             if (!parent.ContainsKey(v) && capacity > 0)
23             {
24                 parent[v] = u;
25                 queue.Enqueue(v);
26                 reachedSink = v.Equals(sink);

```

```

27         }
28     }
29 }
30
31 if (!reachedSink)
32     break;
33
34 double pathFlow = double.PositiveInfinity;
35 T curr = sink;
36 while (!curr.Equals(source))
37 {
38     T prev = parent[curr];
39     pathFlow = Math.Min(pathFlow, residualCapacity[prev][curr]);
40     curr = prev;
41 }
42
43 curr = sink;
44 while (!curr.Equals(source))
45 {
46     T prev = parent[curr];
47     residualCapacity[prev][curr] -= pathFlow;
48     residualCapacity[curr][prev] += pathFlow;
49     curr = prev;
50 }
51
52 maxFlow += pathFlow;
53 }
54
55 return maxFlow;

```

Пример входных данных. Для проверки использовался файл /Users /lunis/Documents/graph9.txt. Граф соответствует примеру со схемы: источник S, сток T, пропускные способности подписаны на дугах.

```

DIRECTED WEIGHTED
S A B C D T
S C 2
S D 2
C A 2
C B 2

```

D A 2
A T 2
B T 3

Пример работы в консоли.

Выберите опцию: 2

Введите путь к файлу: /Users/lunis/Documents/graph9.txt

Граф успешно загружен.

Выберите опцию: 18

Введите вершину-источник (Source): S

Введите вершину-сток (Sink): T

Найдён увеличивающий путь: S -> C -> A -> T с потоком 2

Найдён увеличивающий путь: S -> D -> A -> C -> B -> T с потоком 2

Максимальный поток из S в T: 4

12 Задание на визуализацию

Условие. Творческое задание, включающее визуализацию графов. Направленность может быть обучающей или развлекательной. В данной работе реализовано web-приложение.

Решение. Визуализация реализована как веб-приложение на Blazor. Проект Graph.Web использует общий класс Graph<string>. Через интерфейс можно создавать, импортировать, редактировать и экспортировать графы, а также запускать алгоритмы с пошаговой подсветкой.

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 builder.Services
4     .AddRazorComponents()
5     .AddInteractiveServerComponents();
6
7 var app = builder.Build();
8
9 app.UseStaticFiles();
10 app.UseAntiforgery();
11
12 app.MapRazorComponents<App>()
13     .AddInteractiveServerRenderMode();
14
15 app.Run();
```

На странице приложения хранится текущий граф, координаты вершин, выбранные вершины, подсветка рёбер и шаги алгоритма.

```
1 protected Graph<string> _graph = default!;
2 protected Graph<string> Graph => _graph;
3
4 private readonly Dictionary<string, GraphPoint> _positions =
    ↪ new(StringComparer.Ordinal);
5 private readonly HashSet<string> _visitedVertices = new(StringComparer.Ordinal);
6 private readonly HashSet<GraphEdgeKey> _committedEdges = new();
7 private HashSet<string> _highlightVertices = new(StringComparer.Ordinal);
8 private HashSet<GraphEdgeKey> _highlightEdges = new();
```

```
9 private List<GraphStep> _stepSessionSteps = new();
10 private int _stepIndex = -1;
```

Добавление вершин и рёбер в web-интерфейсе выполняется через методы страницы, которые вызывают методы класса графа.

```
1 protected void AddVertexFromPanel()
2 {
3     if (string.IsNullOrEmpty(_newVertexName))
4     {
5         ShowError("Введите имя вершины.");
6         return;
7     }
8
9     if (!_graph.AddVertex(_newVertexName))
10    {
11        ShowError($"Вершина '{_newVertexName}' уже существует.");
12        return;
13    }
14
15    _positions[_newVertexName] = SpawnPositionNearCenter();
16    SetStatus($"Вершина '{_newVertexName}' добавлена.");
17    _newVertexName = "";
18 }
19
20 protected void AddEdgeFromPanel()
21 {
22     if (!HasPairSelection)
23     {
24         ShowError("Выбери две вершины по порядку на графе.");
25         return;
26     }
27
28     var weight = ParseWeight(_edgeWeightInput);
29     _graph.AddEdge(PrimarySelection!, SecondarySelection!, weight);
30     SetStatus($"Рёбро '{PrimarySelection} -> {SecondarySelection}' добавлено.");
31 }
```

Алгоритмы запускаются из интерфейса. Через делегат `log` они передают в приложение текстовые шаги. Эти шаги затем используются для подсветки вершин и рёбер.

```

1 protected async Task RunMaxFlow()
2 {
3     if (!HasPairSelection)
4     {
5         ShowError("Выбери две вершины на графе: source, затем sink.");
6         return;
7     }
8
9     if (!_graph.IsDirected || !_graph.IsWeighted)
10    {
11        ShowError("Edmonds-Karp требует ориентированный взвешенный граф.");
12        return;
13    }
14
15    var maxFlow = await RunWithSteps(
16        log => _graph.GetMaxFlowEdmondsKarp(PrimarySelection!,
    ↪ SecondarySelection!, log));
17
18    _resultTitle = "Edmonds-Karp";
19    _resultText = $"Максимальный поток: {maxFlow:0.###}";
20 }

```

Для отображения графа используется SVG-разметка. Вершины выводятся окружностями, рёбра — линиями, а для ориентированных графов добавляется маркер стрелки.

```

1 <svg class="graph-canvas"
2     @ref="_svgRef"
3     viewBox="@SvgViewBox"
4     @onpointermove="OnPointerMove"
5     @onpointerup="OnPointerUp"
6     @onpointerdown="OnCanvasPointerDown">
7     <defs>
8         <marker id="arrow" markerWidth="11" markerHeight="11" refX="9"
    ↪ refY="5.5" orient="auto">
9             <path d="M 0 0 L 11 5.5 L 0 11 z" fill="#ffd1a1"></path>
10        </marker>
11    </defs>
12
13    @foreach (var edge in EdgeVisuals)

```

```

14    {
15        <line x1="@edge.X1" y1="@edge.Y1"
16            x2="@edge.X2" y2="@edge.Y2"
17            class="@edge.CssClass"
18            marker-end="@(_graph.IsDirected ? "url(#arrow)" : null)"></line>
19    }
20 </svg>

```

Небольшой JavaScript-модуль используется для скачивания экспортированного графа, обработки горячих клавиш и преобразования координат курсора в координаты SVG.

```

1 window.graphInterop = {
2     downloadText(fileName, text) {
3         const blob = new Blob([text], { type: "text/plain;charset=utf-8" });
4         const url = URL.createObjectURL(blob);
5         const link = document.createElement("a");
6         link.href = url;
7         link.download = fileName;
8         document.body.appendChild(link);
9         link.click();
10        link.remove();
11        URL.revokeObjectURL(url);
12    },
13
14    getSvgPoint(svg, clientX, clientY) {
15        const point = svg.createSVGPoint();
16        point.x = clientX;
17        point.y = clientY;
18        return point.matrixTransform(svg.getScreenCTM().inverse());
19    }
20 };

```

В результате практической работы получена единая программа: консольная часть подходит для проверки алгоритмов, а web-интерфейс позволяет визуально редактировать графы и наблюдать выполнение алгоритмов по шагам.



Рисунок 1 – Внешний вид web-приложения для визуализации графов